

Отчёт по HW2 “Протокол ARP. Адресация уровня Data Link (OSI)”

ФИО: Александр Васюков

Группа: БПИ-235

Email: avvasiukov@edu.hse.ru

Декларация об использовании ИИ

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

Цель работы

Реализовать консольную утилиту на Go для практической работы с протоколом ARP на уровне Ethernet-кадров:

- захват ARP-пакетов в promiscuous-режиме,
- определение MAC-адреса роутера через ARP request/reply,
- сбор статистики сетевого трафика по метрикам за заданное время.

Используемые технологии

- Язык: Go 1.24
- Низкоуровневый PCAP API: github.com/miekg/pcap
- Анализ трафика: Wireshark
- ОС тестирования: macOS

Структура проекта

```
HW-02/
├── main.go           // точка входа
└── internal/
    ├── arp/
    │   ├── frame.go // сериализация Ethernet и ARP
    │   └── commands/
    │       ├── command.go // общий интерфейс команд и env
    │       ├── sniff.go   // захват ARP
    │       ├── discover.go // поиск MAC роутера
    │       └── stats.go   // сбора метрик
    └── sysinfo/
        └── netinfo.go    // интерфейс, IPv4, default gateway
```

Каждое подзадание вынесено в отдельный файл `internal/commands/*.go`.

Запуск

```
go run . -iface en0
```

При старте утилита выводит список поддерживаемых команд:

- sniff,
- discover,
- stats,
- help,
- exit.

Задача 1. Захват ARP-пакетов

Реализация

Команда: sniff [seconds] (файл internal/commands/sniff.go).

- Открывается интерфейс через `pcap.OpenLive(..., promisc=true, ...)`.
- На уровне ядра ставится BPF-фильтр `arp`.
- Каждый кадр разбирается вручную:
 - Ethernet заголовок (DstMAC, SrcMAC, EtherType),
 - ARP заголовок (Operation, sender/target MAC и IP).

Фрагмент вывода

```
arp> sniff 30
Sniffer запущен на en0 на 30s
[2026-03-03T13:53:30.159811+03:00] e8:b7:48:a1:e3:c0 -> ff:ff:ff:ff:ff:ff | ARP
request | 0.0.0.0(be:e7:58:64:57:5e) -> 0.0.0.0(ff:ff:ff:ff:ff:ff)
[2026-03-03T13:53:42.643927+03:00] a4:83:e7:82:f6:ea -> b2:a9:37:a3:49:15 | ARP
request | 10.110.184.72(a4:83:e7:82:f6:ea) -> 10.110.184.199(b2:a9:37:a3:49:15)
[2026-03-03T13:53:42.644044+03:00] b2:a9:37:a3:49:15 -> a4:83:e7:82:f6:ea | ARP
reply | 10.110.184.199(b2:a9:37:a3:49:15) -> 10.110.184.72(a4:83:e7:82:f6:ea)
Sniffer завершен по таймеру
```

В выводе есть targeted ARP и широковещательные ARP-запросы.

Задача 2. Определение MAC-адреса роутера

Реализация

Команда: discover [router-ip] [timeout-sec] (файл internal/commands/discover.go).

1. Определяются MAC и IPv4 локального интерфейса.
2. Формируется ARP request вручную (RFC 826):
 - Operation = 1,
 - SenderMAC/SenderIP = адреса локального интерфейса,
 - TargetMAC = 00:00:00:00:00:00,
 - TargetIP = IP роутера.
3. ARP payload упаковывается в Ethernet frame с DstMAC = ff:ff:ff:ff:ff:ff, EtherType = 0x0806.
4. Кадр отправляется через `pcap.Inject`.
5. Ожидается соответствующий ARP reply.

Фрагмент вывода

```
arp> discover
Интерфейс: en0
Локальный IP: 10.110.184.199
Локальный MAC: b2:a9:37:a3:49:15
Router IP: 10.110.184.1
Router MAC: 00:00:0c:07:ac:1e
```

Задача 3. Сбор статистики

Реализация

Команда: stats <seconds> [router-ip] (файл internal/commands/stats.go).

- Сначала определяется MAC роутера.
- Далее в течение заданного времени читаются кадры и считаются 8 метрик.
- Для устойчивого появления ARP-обмена в сети отправляется probe ARP request к роутеру в начале измерения.

Фрагмент вывода

```
arp> stats 10
Сбор статистики на en0 в течение 10s
Устройство <-> роутер: 10.110.184.199 (b2:a9:37:a3:49:15) <-> 10.110.184.1
(00:00:0c:07:ac:1e)
Отправлен probe ARP request к роутеру 10.110.184.1 для активации ARP-обмена.

Результаты сбора статистики:
1. Всего Ethernet фреймов: 43
2. Всего ARP пакетов: 2
3. Уникальных MAC адресов: 10
4. Широковещательных Ethernet сообщений: 1
5. Широковещательных ARP сообщений: 1
6. Gratuitous ARP Requests: 0
7. Пары ARP targeted request + response: 1
8. Объем данных между устройством и роутером (байт): 1956
Период наблюдения: 10s
```

Способ подсчёта метрик

1. **Всего Ethernet фреймов** — счётчик всех успешно прочитанных кадров.
2. **ARP пакеты** — кадры с EtherType == 0x0806.
3. **Уникальные MAC** — множество srcMAC и dstMAC.
4. **Broadcast Ethernet** — dstMAC == ff:ff:ff:ff:ff:ff.
5. **Broadcast ARP** — ARP-кадры с broadcast dstMAC.
6. **Gratuitous ARP Requests** — ARP request, где senderIP == targetIP.
7. **Targeted request+response пары** — сопоставление request/reply по IP-адресам в правильном направлении.
8. **Объём данных между устройством и роутером** — сумма pkt.Len полная длина кадра из PCAP, включая padding только для кадров между MAC устройства и MAC роутера.

Скриншоты

```
alexvasyukov@Alexanders-MacBook-Air HW-02 % go run . -iface en0
Интерфейс: en0
Доступные команды:
  help           - показать список команд
  sniff [seconds] - захват ARP-пакетов в promiscuous режиме
  discover [router-ip] [timeout-sec] - узнать MAC-адрес роутера через ARP
  stats <seconds> [router-ip] - сбор 8 метрик по Ethernet/ARP за заданное время
  exit | quit    - завершить программу
arp> sniff
Sniffer запущен на en0 на 30s
Формат вывода: [timestamp] src-mac -> dst-mac | ARP op | sender-ip(sender-mac) -> target-ip(target-mac)
[2026-03-03T13:53:30.159811+03:00] e8:b7:48:a1:e3:c0 -> ff:ff:ff:ff:ff:ff | ARP request | 0.0.0.0(08:00:00:00:00:00) -> 0.0.0.0(08:00:00:00:00:00)
[2026-03-03T13:53:42.643927+03:00] a4:83:e7:82:f6:ea -> b2:a9:37:a3:49:15 | ARP request | 10.110.184.72(a4:83:e7:82:f6:ea) -> 10.110.184.199(b2:a9:37:a3:49:15)
[2026-03-03T13:53:42.644044+03:00] b2:a9:37:a3:49:15 -> a4:83:e7:82:f6:ea | ARP reply | 10.110.184.199(b2:a9:37:a3:49:15) -> 10.110.184.72(a4:83:e7:82:f6:ea)
Sniffer завершен по таймеру
arp> discover
Интерфейс: en0
Локальный IP: 10.110.184.199
Локальный MAC: b2:a9:37:a3:49:15
Router IP: 10.110.184.1
Router MAC: 00:00:0c:07:ac:1e
arp> stats 10
Сбор статистики на en0 в течение 10s
Устройство <=> роутер: 10.110.184.199 (b2:a9:37:a3:49:15) <=> 10.110.184.1 (00:00:0c:07:ac:1e)
Отправлен probe ARP request к роутеру 10.110.184.1 для активации ARP-обмена.

Результаты сбора статистики:
1. Всего Ethernet фреймов: 43
2. Всего ARP пакетов: 2
3. Уникальных MAC адресов: 10
4. Широковещательных Ethernet сообщений: 1
5. Широковещательных ARP сообщений: 1
6. Gratuitous ARP Requests: 0
7. Пары ARP targeted request + response: 1
8. Объем данных между устройством и роутером (байт): 1956
Период наблюдения: 10s
arp> exit
Выход
alexvasyukov@Alexanders-MacBook-Air HW-02 %
```

Figure 1: Демонстрация работы

Ответы на вопросы

Почему количество уникальных MAC-адресов ограничено?

ARP работает в пределах L2-сегмента и не маршрутизируется через роутер. Также ARP-кэш снижает частоту запросов. Поэтому за короткое окно виден ограниченный набор активных соседей.

Как рассчитывается объём трафика между устройством и роутером?

Проверяется, что кадр идёт либо host -> router, либо router -> host по MAC-адресам. Если условие истинно, прибавляется pkt.Len из PCAP.

Для чего нужны ARP targeted request/response?

Targeted ARP request нужен для получения MAC по известному IP в локальной сети перед отправкой L3-трафика.

- Request (Operation=1): broadcast, TargetMAC=00:00:00:00:00:00, TargetIP=искомый.
- Reply (Operation=2): unicast, содержит найденный MAC и возвращается инициатору.

Что такое Gratuitous ARP Request?

Это ARP request, в котором SenderIP == TargetIP (узел спрашивает сам себя). Используется для обновления ARP-кэшей соседей и обнаружения конфликтов IP.

Вывод

Разработана интерактивная утилита на Go для низкоуровневой работы с ARP без высокоуровневых парсеров пакетов. Реализованы все требуемые режимы: захват ARP, определение MAC роутера и сбор 8 метрик. В ходе тестирования подтверждён корректный ARP обмен между устройством и роутером 10.110.184.1, а также работоспособность статистики в реальной сети.